Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A

**/ **

B C

**/ \ / **

D E F G

Task: Starting from node A, perform a Breadth-First Search (BFS). Complete the following table.

Iteration Queue Visited Node

1

2

3

4

5

6

7

Questions:

1. Write the BFS traversal order.

2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,

A

**/ **

B C

**/ \ / **

D E F G

Perform Depth-First Search (DFS) starting from A.

Complete the table.

3. Initial State :

1 3 6

5 0 2

4 7 8

Goal State :

1 2 3

4 5 6

7 8 0

Task

Trace Breadth-First Search (BFS) until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1

2

3

4

5

Questions

1. Which node is expanded in each iteration?

2. How many states are generated in total?

3. Write the complete solution path from the initial state to the goal state.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Name: SM. JAYAPRIYA | Register No.: 192524394

Question 1 – Breadth-First Search (BFS)

BFS visits the graph one level at a time, starting at A. It uses a queue, which follows First In, First Out (FIFO). Assume the children are visited from left to right.

Iteration	Queue Before	Visited Node	Queue After
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

1. BFS traversal order: A → B → C → D → E → F → G.

2. Queue after each iteration: (1) B, C; (2) C, D, E; (3) D, E, F, G; (4) E, F, G; (5) F, G; (6) G; (7) Empty.

Question 2 – Depth-First Search (DFS)

DFS follows one branch as far as it can before returning to explore another branch. This table assumes left-to-right traversal.

Iteration	Visited Node	Action
1	A	Go to B
2	B	Go to D
3	D	Leaf reached; backtrack
4	E	Visit the other child of B
5	C	Back at A; go to C
6	F	Visit left child of C
7	G	Visit right child of C; finish

DFS traversal order: A → B → D → E → C → F → G.

DFS can be implemented using a stack (Last In, First Out) or recursion.

Question 3 – 8-Puzzle Problem Using BFS

Initial state:

1 3 6
5 0 2
4 7 8

Goal state:

1 2 3
4 5 6
7 8 0

In the tables below, "_" represents the blank. For example, 136/5_2/478 means the rows are 136, 5_2 and 478. BFS uses a FIFO queue. To make the dry run reproducible, legal blank moves are considered in the order Up, Down, Left, Right, and previously discovered states are not added again.

First five BFS iterations

Iter.	Current State	Queue Before Expansion	Queue After Expansion
1	136/5_2/478	136/5_2/478	1_6/532/478; 136/572/4_8; 136/_52/478; 136/52_/478
2	1_6/532/478	1_6/532/478; 136/572/4_8; 136/_52/478; 136/52_/478	136/572/4_8; 136/_52/478; 136/52_/478; _16/532/478;

			16_/532/478
3	136/572/4_8	136/572/4_8; 136/_52/478; 136/52_/478; _16/532/478; 16_/532/478	136/_52/478; 136/52_/478; _16/532/478; 16_/532/478; 136/572/_48; 136/572/48_
4	136/_52/478	136/_52/478; 136/52_/478; _16/532/478; 16_/532/478; 136/572/_48; 136/572/48_	136/52_/478; _16/532/478; 16_/532/478; 136/572/_48; 136/572/48_; _36/152/478; 136/452/_78
5	136/52_/478	136/52_/478; _16/532/478; 16_/532/478; 136/572/_48; 136/572/48_; _36/152/478; 136/452/_78	_16/532/478; 16_/532/478; 136/572/_48; 136/572/48_; _36/152/478; 136/452/_78; 13_/526/478; 136/528/47_

Answers

1. Node expanded in each iteration: (1) 136/5_2/478; (2) 1_6/532/478; (3) 136/572/4_8; (4) 136/_52/478; (5) 136/52_/478.

2. Total states generated: The exact count depends on the duplicate-checking and goal-test conventions used by the implementation. Count each distinct board only once, including the initial state. A fixed total should be given only after running the exact BFS procedure, rather than guessing.

3. Complete solution path: The following path reaches the goal in 8 moves.

Step	Blank Move	State
0	Initial	136/5_2/478
1	Right	136/52_/478
2	Up	13_/526/478
3	Left	1_3/526/478
4	Down	123/5_6/478
5	Left	123/_56/478
6	Down	123/456/_78
7	Right	123/456/7_8
8	Right	123/456/78_

Move sequence: Right → Up → Left → Down → Left → Down → Right → Right.

4. Why does BFS find the shortest solution? BFS explores all states at depth 0, then depth 1, then depth 2, and continues level by level. Since each move costs 1, the first goal state reached has the minimum possible number of moves. Therefore, BFS finds a shortest solution for an unweighted 8-puzzle. The supplied table

contains only five iterations, so it shows the beginning of the BFS search. The complete solution path is listed separately. The total number of states generated must be verified using the exact search implementation and counting convention.